

Research Notes

Good Practices for Applied Economics Research

Personal notes on research culture, reproducibility, coding, communication, and professional development, written while learning to do (and support) careful empirical work

Gabriela Noemi Villalba Marecos

Department of Economics, University of Sussex

Version 1.0 | August 2026

Contents

1	Why I Keep These Notes	3
2	A Few Principles I Try to Follow	3
2.1	Accuracy before speed	3
2.2	Reproducibility by default	3
2.3	Transparency and intellectual honesty	3
2.4	Substantive reasoning, not just execution	3
2.5	Continuous learning	3
3	The Kind of Projects This Applies To	4
4	What Good RA Work Looks Like	4
5	Communication Habits Worth Keeping	5
5.1	Planning before starting	5
5.2	Progress updates that actually say something	5
5.3	Asking better questions	5
6	Project Organisation	5
6.1	File naming	6
7	Reproducible Research	6
7.1	The master file	6
7.2	No undocumented manual changes	7
7.3	Logs and checks	7
8	Coding Standards I Try to Hold Myself To	7
8.1	General principles	7
8.2	Comments that explain, not narrate	8
8.3	Temporary files and preservation	8
8.4	Avoiding repeated code	8
8.5	Python	8
9	Data Management	8
9.1	Raw data	8
9.2	Data dictionaries	9
9.3	Identifiers	9
9.4	Merges	9
9.5	Confidential and personal data	9
10	Literature Reviews	9
10.1	Finding papers	10
10.2	Reading papers	10
10.3	A literature matrix	10
11	Tables, Figures, and Research Outputs	10
11.1	Tables	10
11.2	Figures	10
11.3	A quality check before sending anything out	11
12	Using AI Tools Responsibly	11

12.1 Where AI genuinely helps	11
12.2 Where I try to be careful	11
12.3 The verification rule	11
13 Research Integrity, Ethics, and Authorship	12
13.1 Errors	12
13.2 Plagiarism and source use	12
13.3 Authorship and acknowledgement	12
14 Reviewing My Own Work	12
14.1 Self-review	12
14.2 Getting reviewed by someone else	12
15 How I've Approached Ramping Up on a New Project	13
16 Checklists I Actually Use	13
17 Professional Development	14
18 Learning Resources and a Personal Log	14
19 Closing Note	15

1 Why I Keep These Notes

These are my own working notes on what makes empirical research reliable, reproducible, and easy for someone else (including my future self) to pick up. I started writing them down after enough small, preventable mistakes — an unmatched merge I didn't investigate, a table I couldn't reproduce two weeks later, a script with no header explaining what it did. None of these mistakes were dramatic on their own, but together they taught me that good research habits are not automatic; they have to be practised deliberately.

I am writing this both as a research assistant and as someone who is beginning to think about what I would want from an RA if I were supervising one. The principles below are not specific to a single project or role. They apply whenever I am cleaning data, writing code, reading literature, or producing a table, regardless of whether I am doing the task myself or reviewing it for someone else.

The idea I keep coming back to: I don't write code only for the computer. I write it for the next person who will read it — including myself, months later, when I no longer remember why I made a particular choice. Every dataset should be reproducible, every result traceable, and every important decision documented.

This is a living document. I expect to revise it as I learn more, take on new projects, and pick up better tools.

2 A Few Principles I Try to Follow

2.1 Accuracy before speed

Fast work only helps if it is reliable. I try not to hide uncertainty, force code to produce a result I expect, or make silent assumptions. When something is unclear, the better habit is to write down the uncertainty and ask, rather than guess quietly and move on.

2.2 Reproducibility by default

Someone else — or I, later — should be able to understand how raw data became a table, figure, or regression result. That means data transformations, sample restrictions, variable definitions, and estimation choices need to live in code and supporting notes, not just in my head.

2.3 Transparency and intellectual honesty

Unexpected or null results are still results. Errors should be reported as soon as I notice them and corrected openly. Finding a mistake is a normal part of research; the part to avoid is sitting on it.

2.4 Substantive reasoning, not just execution

Running commands is not the same as doing research assistance well. I try to connect every task back to the underlying research question: what does this variable represent, why is this sample restriction here, how does this output feed into the paper.

2.5 Continuous learning

These skills develop through practice, feedback, and revision. I don't expect to know everything at the start of a project, but I do try to learn from corrections and avoid repeating the same

preventable error twice.

3 The Kind of Projects This Applies To

Most of what follows comes out of working in applied microeconomics, particularly around education and higher education quality, labour markets, inequality, and development. A few recurring themes:

- education and higher education quality;
- educational mismatch and student allocation;
- cooperatives and development;
- labour-market and policy evaluation;
- revising and preparing earlier work for publication;
- communicating research clearly, including through writing and project documentation.

Some of the data I work with is confidential or restricted. A basic rule I hold myself to: data, code, outputs, and draft papers don't leave the approved environment, and they never go into an external AI system or personal cloud storage without explicit authorisation.

4 What Good RA Work Looks Like

Over time I've come to think of good performance not just as "tasks completed," but as:

- reliable and transparent work;
- careful documentation;
- timely, specific communication;
- asking informed questions rather than staying quiet or guessing;
- paying attention to what a result actually means, not just whether the code ran;
- visible improvement over time;
- respect for confidentiality and research ethics.

Typical tasks that fall under this, in my experience:

- locating and summarising academic literature;
- cleaning and documenting datasets;
- writing or reviewing Stata and Python code;
- checking tables, figures, and regression outputs;
- preparing replication files;
- formatting outputs for submission;
- keeping project trackers and documentation up to date.

5 Communication Habits Worth Keeping

5.1 Planning before starting

Before I start a task, I try to be clear with myself about: the project it belongs to, the objective, the expected output, the relevant files, the deadline, and any known constraints. A task doesn't really feel "done" until the output is saved in the right place and documented well enough that someone else could pick it up.

5.2 Progress updates that actually say something

A useful update explains what was completed, what remains unfinished, what problems came up, and what decision or support is needed. I try to avoid vague notes like "worked on the data" and aim instead for something like:

I completed the merge between the institution and programme files. Of 1,245 programme-year observations, 1,198 matched successfully. Forty-seven unmatched observations remain, mainly because institution identifiers changed across years. I saved a list of unmatched cases and need guidance on whether to harmonise these identifiers manually.

5.3 Asking better questions

Before asking a technical question, it's worth making a genuine attempt to investigate first. A strong question usually includes: the objective, the relevant code or file, the error or unexpected result, what I've already tried, and my current best guess at what's going on.

Some things are never worth delaying, though — anything touching confidentiality, data loss, research ethics, or a potentially serious coding error should be raised immediately, not batched into a weekly update.

6 Project Organisation

A consistent folder structure saves a surprising amount of time later. The version I keep coming back to:

```
1 project_name/  
2 |-- 00_admin/  
3 |-- 01_raw_data/  
4 |-- 02_intermediate_data/  
5 |-- 03_clean_data/  
6 |-- 04_code/  
7 |   |-- 00_master.do  
8 |   |-- 01_import.do  
9 |   |-- 02_clean.do  
10 |   |-- 03_construct.do  
11 |   |-- 04_analysis.do  
12 |   |-- 05_tables.do  
13 |   '-- 06_figures.do  
14 |-- 05_outputs/  
15 |   |-- tables/  
16 |   '-- figures/  
17 |-- 06_documentation/  
18 |-- 07_paper/  
19 '-- README.md
```

The exact folder names vary across projects, but the underlying distinction should hold:

- raw data are never overwritten;
- intermediate data are reproducible from raw data;
- clean or analytical data are reproducible from intermediate data;
- tables and figures are produced by code;
- documentation explains decisions that code alone can't capture.

6.1 File naming

File names should be descriptive, stable, and machine-readable.

Prefer:

- `programme_panel_clean.dta`
- `table_2_main_results.tex`
- `figure_3_event_study.pdf`

Avoid:

- `final.dta`
- `final2.dta`
- `newfinalREALLYFINAL.dta`
- file names with unexplained initials or spaces.

Dates: YYYY-MM-DD when they matter.

7 Reproducible Research

Reproducibility, in practice, means the final outputs of a project can be regenerated from the underlying data and code with minimal manual intervention.

7.1 The master file

Every empirical project I work on has (or should have) a master script that runs everything in the right order. In Stata this is usually `00_master.do`.

```
1 clear all
2 set more off
3 version 18.5
4
5 * Define project root
6 * global root "C:/path/to/project"
7
8 * Create data
9 do "${root}/04_code/01_import.do"
10 do "${root}/04_code/02_clean.do"
11 do "${root}/04_code/03_construct.do"
12
13 * Produce results
```

```

14 do "${root}/04_code/04_analysis.do"
15 do "${root}/04_code/05_tables.do"
16 do "${root}/04_code/06_figures.do"

```

Anyone new to the project should be able to understand its order just by opening this file.

7.2 No undocumented manual changes

I try not to manually edit analytical datasets, regression outputs, tables, or figures when the same change could be made reproducibly through code. Manual edits create hidden steps that make errors hard to find later. When a manual step really is unavoidable, it goes in the README or a processing log — not just in my memory.

7.3 Logs and checks

Important scripts should write log files, and code should check its own assumptions: uniqueness of identifiers, expected sample sizes, valid value ranges.

```

1 isid institution_id year
2 assert year >= 2010 & year <= 2026
3 assert inrange(quality_score, 0, 6) if !missing(quality_score)
4 count if missing(institution_id)

```

8 Coding Standards I Try to Hold Myself To

8.1 General principles

Code should be readable, modular, and easy to audit — not just by me, but by anyone reviewing it. Ideally a reader understands not only what the code does, but why.

I try to start every script with a short header:

- project name;
- purpose of the file;
- author;
- date created or updated;
- inputs;
- outputs.

```

1 /*****
2 Project:      Higher Education Quality
3 File:         02_clean_staff.do
4 Purpose:     Clean institution-level academic staff data
5 Author:      Gabriela Villalba Marecos
6 Updated:     2026-08-04
7 Inputs:      staff_raw_2018_2023.xlsx
8 Outputs:     staff_clean.dta
9 *****/

```


8.2 Comments that explain, not narrate

Comments should explain logic, assumptions, and non-obvious decisions — not just restate the command.

Weak comment:

```
1 * Generate ratio
2 gen student_staff_ratio = students / staff
```

Better:

```
1 * Construct students per academic staff member. Institutions with zero
2 * reported staff are set to missing because the ratio is undefined.
3 gen student_staff_ratio = students / staff if staff > 0
```

8.3 Temporary files and preservation

Use temporary files when an intermediate dataset doesn't need to be kept permanently. In Stata, `tempfile`, `preserve`, and `restore` are worth using carefully and deliberately, not as a reflex.

8.4 Avoiding repeated code

When I catch myself repeating the same operation many times, that's usually a sign to use a loop, a local macro, or a reusable program. That said, compact code isn't automatically better — readability still comes first.

8.5 Python

For Python, I aim for clear function names, type hints where they help, and a proper `main` execution block. Hard-coded paths are worth avoiding from the start.

```
1 from pathlib import Path
2 import pandas as pd
3
4
5 def load_programme_data(path: Path) -> pd.DataFrame:
6     """Load and validate programme-level data."""
7     data = pd.read_csv(path)
8     required = {"institution_id", "programme_id", "year"}
9     missing = required.difference(data.columns)
10    if missing:
11        raise ValueError(f"Missing required columns: {sorted(missing)}")
12    return data
13
14
15 if __name__ == "__main__":
16     root = Path(__file__).resolve().parents[1]
17     df = load_programme_data(root / "01_raw_data" / "programmes.csv")
```

9 Data Management

9.1 Raw data

Raw data is read-only, full stop. I don't overwrite, rename, or edit raw files unless a change has been approved and documented, and I try to preserve the original file names from the data provider.

9.2 Data dictionaries

Every project benefits from a data dictionary with:

- variable name;
- variable label;
- unit of observation;
- coding and value labels;
- source;
- transformations;
- known limitations.

9.3 Identifiers

Before merging datasets, it's worth checking:

- whether the identifier is unique;
- whether it changes over time;
- whether leading zeros are meaningful;
- whether it's stored consistently as numeric or string;
- whether missing identifiers have a substantive explanation.

9.4 Merges

Never drop unmatched observations automatically without looking into them. I try to report merge results and save a list of unmatched cases whenever they could plausibly affect the analysis.

```
1 merge 1:1 institution_id year using "staff_clean.dta"
2 tab _merge
3 preserve
4     keep if _merge != 3
5     export excel using "unmatched_merge_cases.xlsx", firstrow(variables)
6     replace
7 restore
```

9.5 Confidential and personal data

Confidential data stays in approved locations only — never personal cloud services, messaging apps, public repositories, or external AI systems. I try to use only the minimum data necessary and strip direct identifiers whenever they aren't needed for the task.

10 Literature Reviews

A literature review should clarify the research question, the theoretical mechanisms, the empirical strategy, the contribution, and how to interpret results — not just list disconnected summaries.

10.1 Finding papers

I try to prioritise:

- peer-reviewed journal articles;
- recognised working-paper series;
- official policy and institutional documents;
- recent papers citing foundational studies;
- systematic reviews when they exist.

For substantial reviews, it's worth recording search terms, databases, and screening decisions along the way.

10.2 Reading papers

For each relevant paper, I try to note: the research question, the setting and institutional context, the data and unit of analysis, the identification strategy, the main findings, the mechanisms or interpretation, the limitations, and how it's relevant to my project.

10.3 A literature matrix

A literature matrix is often more useful than narrative notes in the early stages. Columns I find useful:

Citation	Setting/Data	Method	Main finding	Relevance to project
Author (Year)	Country, sample, period	FE, IV, DiD, RCT, etc.	One concise statement	Contribution, mechanism, or comparison

I try not to cite a paper based only on its abstract when a claim I'm making depends on the full study.

11 Tables, Figures, and Research Outputs

Tables and figures should communicate a clear message and be reproducible from code.

11.1 Tables

A good table usually has: a descriptive title, clearly labelled variables and columns, units where relevant, sample size, standard errors and significance where appropriate, a note explaining the sample and specification, and a source note when needed. I try to avoid overcrowded tables — a reader should be able to follow the main comparison without decoding rows of abbreviations.

11.2 Figures

Figures should have an informative title or caption, labelled axes, units, a readable legend, an explanatory note, and formatting consistent with the rest of the paper. I avoid editing figures manually after export unless the edit is documented and reproducible.

11.3 A quality check before sending anything out

Before I consider an output finished, I check that: sample sizes match the analytical data, coefficients and standard errors match the saved output, variable labels are correct, notes accurately describe the specification, decimal places are consistent, and the output actually comes from the latest version of the code.

12 Using AI Tools Responsibly

AI tools can support research, but they don't replace judgement, verification, or accountability. Whoever submits the work is still responsible for its accuracy — that's a rule I apply to myself as much as to anyone else.

12.1 Where AI genuinely helps

- explaining a coding error;
- suggesting alternative code structures;
- improving the clarity of documentation;
- drafting non-confidential boilerplate;
- brainstorming search terms or robustness checks;
- translating or clarifying technical language.

12.2 Where I try to be careful

- never uploading confidential, restricted, or personally identifiable data;
- never accepting generated citations without checking they exist;
- never reporting a generated claim as fact without checking a reliable source;
- never using AI-generated code without understanding and testing it;
- never asking AI to fabricate results, interpretations, or evidence;
- never concealing substantial AI assistance when disclosure is expected.

12.3 The verification rule

Anything AI-generated that I actually use gets checked independently. For code, that means testing it on real data and looking at intermediate results. For literature, it means opening and reading the original source. For writing, it means confirming every factual claim is actually supported.

AI can assist with research tasks, but it isn't the source of truth. Data, original documents, verified publications, and transparent analysis are.

13 Research Integrity, Ethics, and Authorship

These follow the ethical and professional standards of the relevant institutions, data providers, funders, and journals — not something to improvise case by case.

13.1 Errors

I try to report errors as soon as I find them, explaining what's affected, how the error was found, the likely source, whether earlier outputs might be wrong, and what's already been done to fix it.

13.2 Plagiarism and source use

Text, code, figures, and ideas from external sources need proper credit. Copying code from public repositories without attribution or checking the licence isn't a shortcut worth taking.

13.3 Authorship and acknowledgement

Doing RA work doesn't automatically imply authorship. Authorship depends on the nature and extent of the intellectual contribution, following disciplinary and journal standards. Substantial contributions are worth documenting clearly, since they may lead to acknowledgement, credit in replication materials, or co-authorship where appropriate — decisions best made transparently and early rather than assumed.

14 Reviewing My Own Work

Anything I consider important gets a review pass before I treat it as finished.

14.1 Self-review

Before calling something done, I try to:

1. restart the software and rerun the relevant script from scratch;
2. actually read the warning and error messages, not just scroll past them;
3. check intermediate data, not only the final output;
4. compare counts and summary statistics against what I expected;
5. remove leftover temporary or exploratory code;
6. confirm outputs are saved in the correct folder;
7. write a short, honest completion note.

14.2 Getting reviewed by someone else

A useful reviewer — and a useful review to receive — should be able to identify: the research objective, the input data, the main decisions made, where the output lives, and what's still unresolved. I try to treat feedback as part of the process rather than as a verdict on me personally, and to address comments systematically, noting what actually changed.

15 How I've Approached Ramping Up on a New Project

This is roughly the sequence that's worked for me when getting up to speed on a new project or a new part of the codebase.

Stage	Focus	What I actually do
Week 1	Orientation	Read the project documentation; understand the research question; set up software and folders; note confidentiality rules.
Week 2	Reproducibility	Run the existing master script; inspect it end to end; identify inputs and outputs; reproduce one table myself.
Week 3	Data work	Clean a small dataset; build a data dictionary; run uniqueness and missingness checks.
Week 4	Literature	Prepare a structured summary of key papers and add them to a literature matrix.
Week 5	Outputs	Reproduce and quality-check a table or figure; verify notes, sample sizes, and formatting.
Week 6	Independent task	Take on a small, well-defined task end to end, with documentation and a written progress note.

The pace obviously depends on prior experience with the project's tools and subject matter.

16 Checklists I Actually Use

Before starting a task

- ☐ I understand the research objective.
- ☐ I know which files I should use.
- ☐ I know the expected output and deadline.
- ☐ I've identified any confidentiality restrictions.
- ☐ I know where to save code, data, and outputs.

Before submitting code

- ☐ The script runs from beginning to end.
- ☐ Paths are portable or clearly defined.
- ☐ Raw data were not modified.
- ☐ Important assumptions are documented.
- ☐ Merge results and sample restrictions were checked.
- ☐ Temporary or exploratory commands were removed.
- ☐ Outputs are saved in the correct folder.

Before submitting a literature review task

- ☐ Every cited paper exists and I've actually opened it.
- ☐ Claims accurately represent the source.
- ☐ I recorded the data, method, and main result.
- ☐ I explained why the paper is relevant.
- ☐ References are complete and consistently formatted.

Before submitting a table or figure

- ☐ The output is generated by code.
- ☐ The sample size is correct.
- ☐ Labels and units are clear.
- ☐ Notes describe the specification accurately.
- ☐ Formatting is consistent with the paper.
- ☐ I can explain the substantive message myself.

17 Professional Development

Part of the point of doing this work carefully is what it builds toward. Areas I keep coming back to, and want to keep developing:

- Stata, Python, and reproducible workflows;
- reading and interpreting empirical papers critically;
- academic writing and presentation;
- CVs and applications;
- postgraduate study and research careers;
- building a track record I can point to through demonstrated work;
- contributing to research outputs where the contribution genuinely warrants it.

The shift I'm trying to make over time is from just completing instructions to understanding the wider project — anticipating quality-control needs, proposing solutions, and explaining results clearly to someone who wasn't in the room when I found them.

18 Learning Resources and a Personal Log

I keep a personal learning log with:

- new commands or techniques;
- common errors and how I solved them;
- substantive concepts learned from each project;
- feedback I've received;

- questions to follow up on.

Areas worth continuing to study:

- data management in Stata and Python;
- causal inference and applied econometrics;
- research design;
- version control with Git and GitHub;
- academic writing and data visualisation;
- research ethics and data protection.

19 Closing Note

Research is cumulative. A well-labelled variable, a merge I actually investigated instead of dropping, or a clearly written note might seem like a small thing in the moment, but these are the details that decide whether a project is still trustworthy months or years later.

The point of keeping these notes is to build a habit — careful work, uncertainty discussed openly rather than hidden, and a standing commitment to keep learning.

Every result should be traceable. Every decision should be defensible. There's always more to learn.